ATLAS SKILLTECH UNIVERSITY · PORTAL 2025–26

Research Publication Portal.

System Architecture & Implementation Blueprint — a beginner-friendly, end-to-end plan for a five-stage institutional research publication workflow.

ISDI ISME uGDX LAW

PLANNING ONLY | NO CODE BUILT | v1.0

Prepared: 19 October 2026 · **Prepared for:** ATLAS SkillTech University

Stack: Next.js (App Router, JavaScript) · Tailwind CSS v4 · PostgreSQL · Prisma · Auth.js

Evidence: 59 local design reference images (51 screens + 8 logo assets). No live Figma API was used.

Scope: architecture, data model, security, roadmap & open questions — **not** an implementation.

Table of Contents

CONFIRMED supported by a reference screen PROPOSED recommended, needs institutional sign-off

1	Executive Summary	§1
2	Project Overview	§2
3	Reference Audit & Screen Inventory	§3
4	Confirmed Requirements vs Assumptions	§4
5	Role-wise Functionality	§5
6	Login & Registration Architecture	§6
7	Complete Paper Lifecycle & Business Rules	§7
8	System Architecture Diagrams	§8
9	Database Design & ER Diagram	§9
10	API & Module Architecture	§10
11	Authentication & Permission Model	§11
12	Security Architecture	§12
13	Responsive Design Strategy	§13
14	Recommended Next.js Folder Structure	§14
15	Development Roadmap	§15
16	Testing Strategy	§16
17	Deployment & Maintenance	§17
18	Risks, Dependencies & Unresolved Questions	§18
19	Glossary of Technical Terms	§19
20	Final Implementation Checklist	§20

How to read this document

This blueprint is written so a non-technical project owner can follow the whole system, while a developer gets enough structure to build it. Plain-English explanations and analogies sit next to tables and diagrams. Companion Markdown files (`system-architecture-summary.md` , `requirements-and-role-matrix.md` , `implementation-roadmap.md` , `open-questions-and-assumptions.md`) carry the same facts in editable form.

1 Executive Summary

The Research Publication Portal is a single web application that carries a student's or faculty member's research paper from first upload all the way to a published volume — through five supervised stages — while guaranteeing that each paper is seen only by the people officially attached to it.

What the system does, in one breath

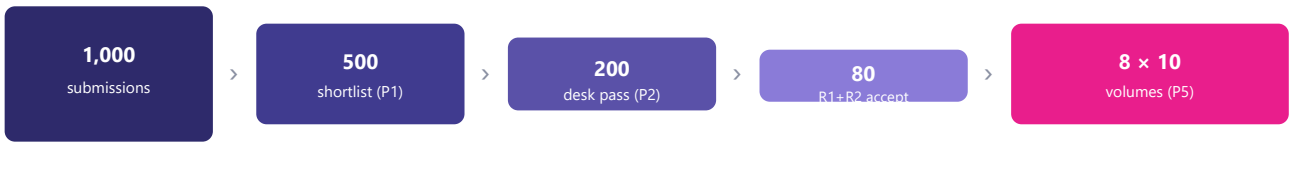
A submitter uploads a paper. A **SPOC** shortlists it by grade. A **Desk Reviewer** runs a plagiarism & AI-writing check. Two **Peer Reviewers** judge it blindly in two rounds. An **Editor** makes the final decision and places accepted papers into published **volumes**. An **Administrator** oversees everyone and assigns who does what — on a paper-by-paper basis.

The one idea that defines the build

> **Per-paper role-based access control**
>
> Everyone registers as a plain **Submitter**. The powerful roles (SPOC, Desk Reviewer, Peer Reviewer, Editor) are attached **to a paper, not to a person**. The same individual can be an author on their own paper and a peer reviewer on someone else's. The system must enforce these boundaries on the **server**, for every request — never by merely hiding a button. The reference designs state this explicitly: "*roles are assigned per paper, never per person*" and "*Nobody can review, grade or edit their own paper.*"

The funnel at a glance

The dashboards describe the scale directly: **"from 1,000 submissions to 8 volumes of 10."**



The publication funnel — targets shown on every dashboard. Numbers are cycle targets, not hard caps.

Headline recommendations

- **Architecture:** a modular Next.js (App Router) application with server-side route handlers and a managed PostgreSQL database — a clean *modular monolith*, not microservices.
- **Security:** one central server-side policy layer enforces per-paper access on every read and write; the API never returns data the caller may not see (true for blind-review confidentiality).
- **Hosting:** a Node-capable host (e.g. Vercel, a container/VPS, or an institutional Node server). **XAMPP/Apache+PHP cannot run a Next.js production server** — it is only the local folder location here.

- **Two prototype shortcuts to revisit for production:** manuscripts are currently referenced by a **Google Drive link** (recommend private object storage), and email is **mocked** (recommend wiring Resend/SendGrid).

> **This is a planning document only**
>
> No application, dependency, database schema or migration has been created. The final section recommends the next phase and asks for approval before any coding begins.

2 Project Overview

Purpose

ATLAS SkillTech University needs a trustworthy, auditable system to manage the annual research publication cycle across its four schools. The portal replaces scattered emails and lost drafts (the login page promises: *"No emails, no lost drafts — just a live status page for your submission"*) with a single tracked pipeline.

The four schools (confirmed)

Each school has its own colour, its own ID prefix, and its own programme taxonomy. The school is **inferred from the Paper-ID prefix** when not supplied.

School	Prefix	Full name / level	Programme taxonomy (examples)
ISDI	`ISDI-`	School of Design & Innovation (B.Des / B.Tech / PG)	Product Design · Communication Design & New Media · Animation VFX & Gaming · Design Futures M.Des
ISME	`ISME-`	BBA (FY/SY/TY/4th year)	ACCA · AIET · DBA · PSY · General BBA
uGDX	`UGDX-`	B.Tech (FY/SY/TY/4th year)	Robotics · AIDS · ITAIC · AIML
LAW	`LAW-`	LLB (FYLLB / SYLLB)	LLB · Bachelor of Laws

Core concepts in plain English

Frontend — what users see and click (the forms, dashboards, tables). Built with Next.js + Tailwind.

Backend — the "back office" that checks the rules before doing anything. Lives inside Next.js on the server.

Database — the organised filing cabinets (PostgreSQL) that store papers, people, reviews and history.

Authentication — checking your identity at the door (who are you?).

Authorization — deciding which cabinets your identity may open (what may you do?).

API — the internal "request slips" passed between the screen and the back office.

Technology stack (fixed by the brief)

Layer	Choice	Note
Framework	Next.js 15+ **App Router**	No Pages Router
Language	**JavaScript only**	No TypeScript files
Styling	**Tailwind CSS v4**	CSS-first `@theme` tokens; no stray CSS files
Responsiveness	Mobile-first	Reusable components, accessible semantic HTML
Database	PostgreSQL + Prisma PROPOSED	Relational integrity + transactions
Auth	Auth.js (NextAuth) PROPOSED	Credentials now; SSO later

3 Reference Audit & Screen Inventory

> **Audit method & honesty statement**
>
> All findings below come from inspecting **59 local PNG images** in `reference/figma-design/` at full resolution (51 UI screens + 8 logo assets; 3 logo variants inspected in full, the rest are size/format duplicates of the same marks). **No Figma URL or connected Figma document was provided**, so no live Figma data was read. No reference file was modified, renamed or deleted. Where a workflow is implied but not shown on a screen, it is marked PROPOSED.

3.1 Screen inventory by role

Area	Screens found	#
Login / Register	Log in · Register (Student) · Register (Faculty)	3
Student (Submitter)	Dashboard · My Submissions · Submit Paper · Activity	4
Faculty (Submitter)	Dashboard · My Submissions · Submit Paper · Activity	4
SPOC	Dashboard · Papers · Import · Paper detail · Papers (role switcher) · Activity · New-paper overlay	7
Desk Reviewer	Dashboard · Papers · Paper detail · Activity	4
Peer Reviewer	Dashboard · Papers · Paper detail · Paper — no access · Papers (role switcher) · Activity	6
Editor	Dashboard · Papers · Paper detail · Volumes · Archive · Activity	6
Admin	Dashboard (+4 per-school +1 filtered) · Papers · Paper detail · Paper access · Import · Volumes · Archive · Users · Activity · New-paper + Assign-role + Assign-role conflict overlays	17
Logo / branding	Full lockup · square app tile · horizontal lockup · variants	8
	Total reference files	**59**

3.2 Shared design-system observations

- **App shell:** left sidebar with brand block ("RESEARCH PUB / 2025–26 Portal"), role-specific nav, and a "Signed in as" card with a coloured role chip + Sign out.
- **Dashboards** share a header greeting, six KPI cards (Total Papers, Phase-1 Shortlist, Desk-Review Passed, Ready for Volumes, Accepted, Rejected) and a five-card "Phase Timeline · 2025".
- **Papers tables** use a consistent column set: `ID · TITLE/STUDENT · SCHOOL · GRADE · PHASE · STATUS · DRIVE` with search + dropdown filters (schools / programmes / phases / statuses).
- **Scope banners** on reviewer/editor screens: *"Showing only papers assigned to you as … Other papers … are not visible to your account. Contact the Administrator if a paper is missing."*
- **Confidentiality notes** appear inline on reviewer screens (see §7, §11).

3.3 Logo assets to preserve

Primary full lockup (teal isometric cube + "ATLAS SKILLTECH UNIVERSITY" + four school tags), a square app tile (indigo background — ideal favicon / PWA icon), and a horizontal lockup. Brand palette: indigo `#2E2A6B`, teal cube, magenta/pink accent; school colours ISDI pink, ISME blue, uGDX red, LAW orange.

3.4 What is NOT in the references (gaps)

PROPOSED and listed as open questions: password reset / forgot-password, email verification screen, account approval/activation, dedicated error/loading screens, reviewer scoring rubric, annotated-file upload, co-author UI, withdrawal flow, double-blind author masking.

4 Confirmed Requirements vs Assumptions

4.1 Confirmed by the references CONFIRMED

Requirement	Evidence (on-screen)
Seven roles: Submitter, SPOC, Desk Reviewer, Peer Reviewer, Editor, Admin (+ base types Student/Faculty)	Login "Enter as" chips; Users "Role Key"
Five-phase pipeline P1–P5 with named owners & targets	Dashboard "Phase Timeline"; paper-detail cards
Roles assigned per paper, never per person	Users subtitle; Assign-role modal "Applies to this paper only"
Paper visible only to submitter + assigned roles + Admin	Paper-access "Visibility" panel; "no access" screen
Conflict block: cannot review/grade/edit own paper	Assign-role conflict modal; Activity "Assignment blocked"
Single-blind review (desk / co-reviewer / editor hidden; author visible)	Reviewer paper-detail confidentiality banner
Desk review via Turnitin: similarity % + AI-writing % + report URLs + notes; Pass/Fail	Desk-reviewer paper detail
Peer review: Accept/Revise/Reject + comments + optional revision link	Peer-reviewer paper detail
Editor final decision + volume assembly (Ready→Placed→Locked→Archived)	Editor Volumes & Archive
Both R1 & R2 "Accept" required to reach "Ready for Volumes"	Volumes board green R1/R2 chips
Manuscript via Google Drive link; email mocked	Submit form; Activity subtitle
Bulk import CSV/Excel upsert on `paper_id`; school from prefix	Import screen
Multi-role accounts with "Viewing as / Switch role"	Role-switcher screens

4.2 Assumptions / proposals needing sign-off PROPOSED

Assumption	Why proposed / what to confirm
PostgreSQL + Prisma + Auth.js stack	Not dictated by brief; recommended for fit & integrity
Private object storage + signed URLs for manuscripts	Replaces/augments public-ish Drive links for security
Admin↔SPOC split of reviewer-assignment authority	Dropdowns appear on both; exact rule unconfirmed
Email verification + password reset flows	No reference screens exist
Revision/resubmission loop detail & round cap	"Revise" exists but no full resubmission UI
Co-author / multi-author support	Single author field only in prototype

Assumption	Why proposed / what to confirm
Withdrawal flow; author-blind (double-blind) option	Not shown; may be institutionally required
Node hosting + managed Postgres (not XAMPP)	Deployment target unconfirmed

Every proposed item is tracked with a priority in `open-questions-and-assumptions.md` and summarised in §18.

5 Role-wise Functionality

Each role has a distinct workspace, a distinct navigation set, and — critically — distinct **permissions enforced on the server**. No two roles share the same powers.

5.1 Navigation by role (confirmed)

Role	Sidebar navigation
Submitter (Student/Faculty)	Dashboard · My Submissions · Submit Paper · Activity
SPOC	Dashboard · Papers · Import · Activity
Desk Reviewer	Dashboard · Papers · Activity
Peer Reviewer	Dashboard · Papers · Activity
Editor	Dashboard · Papers · Volumes · Archive · Activity
Admin	Dashboard · Papers · Import · Volumes · Archive · Users · Activity

5.2 Role × capability matrix (confirmed)

Capability	Sub	SPOC	Desk	Peer	Editor	Admin
Submit / create own paper	✓	✓[1]	✓[1]	✓[1]	✓[1]	✓
See only own papers	✓	–	–	–	–	all
See assigned papers only	–	✓	✓	✓	✓	all
Phase-1 shortlist + set grade	–	✓	–	–	–	✓
Assign Desk/Peer reviewers	–	✓	–	–	–	✓
Assign SPOC/Editor roles	–	–	–	–	–	✓
Record Turnitin / desk Pass·Fail	–	–	✓	–	–	✓
Submit peer decision (R1/R2)	–	–	–	✓	–	✓
See other reviewer's report	–	✓	✗	✗	✓	✓
Final decision (Accept/Revise/Reject)	–	–	–	–	✓	✓
Manage volumes / archive	–	–	–	–	✓	✓
Bulk import (CSV/Excel)	–	✓	–	–	–	✓
Manage people & roles	–	–	–	–	–	✓

[1] Only as a Submitter on a *different* paper — never where they hold a review/grade/edit role (conflict block).

5.3 Role detail cards

Submitter — Student & Faculty

Purpose: create and track own papers. **Can:** submit (Paper ID, school→programme, title, author prefill, batch, guide, themes, Drive link), track status, edit Drive-link/themes *until shortlisting begins*, read the activity feed. **Cannot:** see others' papers, review, grade, or touch volumes. Student and Faculty differ only by greeting, identity card, and the pre-filled author field.

SPOC — "Shortlist + grade assigned papers"

Can: view assigned papers; **Shortlist to Phase 2**; set/confirm Grade; assign Desk & Peer reviewers; create & bulk-import papers; see both peer reports on assigned papers. **Cannot:** shortlist own submission; make the final decision; manage volumes.

Desk Reviewer — "Desk-check assigned papers"

Can: record Turnitin similarity %, AI-writing %, report URLs, notes; set **Pass** → **Phase 3** or **Fail**. **Cannot:** see peer reports or the editor's decision; assign reviewers; finalise; manage volumes.

Peer Reviewer — "Review assigned papers (R1 / R2)"

Can: submit Accept/Revise/Reject + comments (+ optional revision link) on the round they are assigned. **Cannot:** see the desk review, the other reviewer's report, or the editor's decision; review own paper; open unassigned papers.

Editor — "Final decision + volume"

Can: see the full paper (incl. both peer reports); set Accept / Request revision / Reject; build volumes (create, Fill-smartly ≤10, Cover, Lock, recipients, send to publishing); manage Archive (Download PDF, Restore). **Scoped** to papers assigned to them as Editor.

Admin — "Every paper, every person"

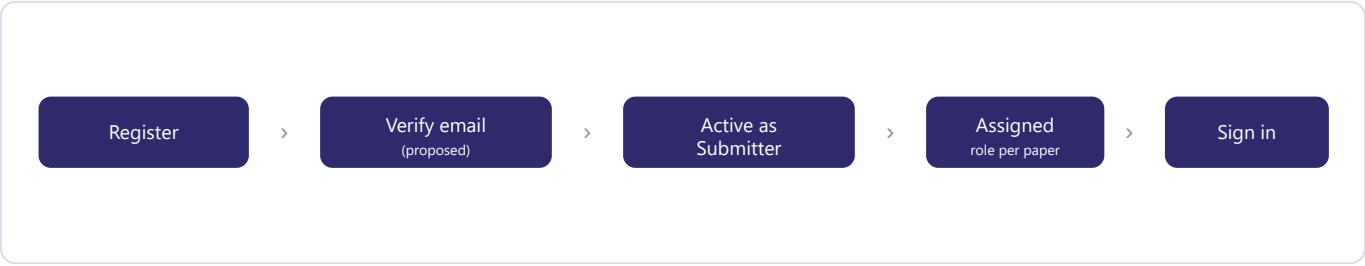
Can: everything, everywhere; People & roles management with a live conflict check and a visibility panel ("N/11 people can open this paper"); see open-role-slot alerts; per-school analytics with deep programme/batch filters.

6 Login & Registration Architecture

6.1 What the references show CONFIRMED

- **Login:** email + password + "Sign in"; a demo "Enter as" panel listing the seven roles; a "First submission? Start here →" link to registration.
- **Register:** an "I am a" **Student / Faculty** toggle; fields Full name, University email, School (dropdown), **Roll No. (students only)**, Password ("*min 6 characters*"). Helper: "*Every account starts as a Submitter. An administrator assigns reviewer, SPOC and editor roles per paper.*"
- The three-step promise: **01 Register → 02 Submit paper → 03 Track status**.

6.2 Account lifecycle (proposed) PROPOSED



Proposed account lifecycle. Email verification & approval are not in the references and need confirmation.

6.3 Security requirements for auth

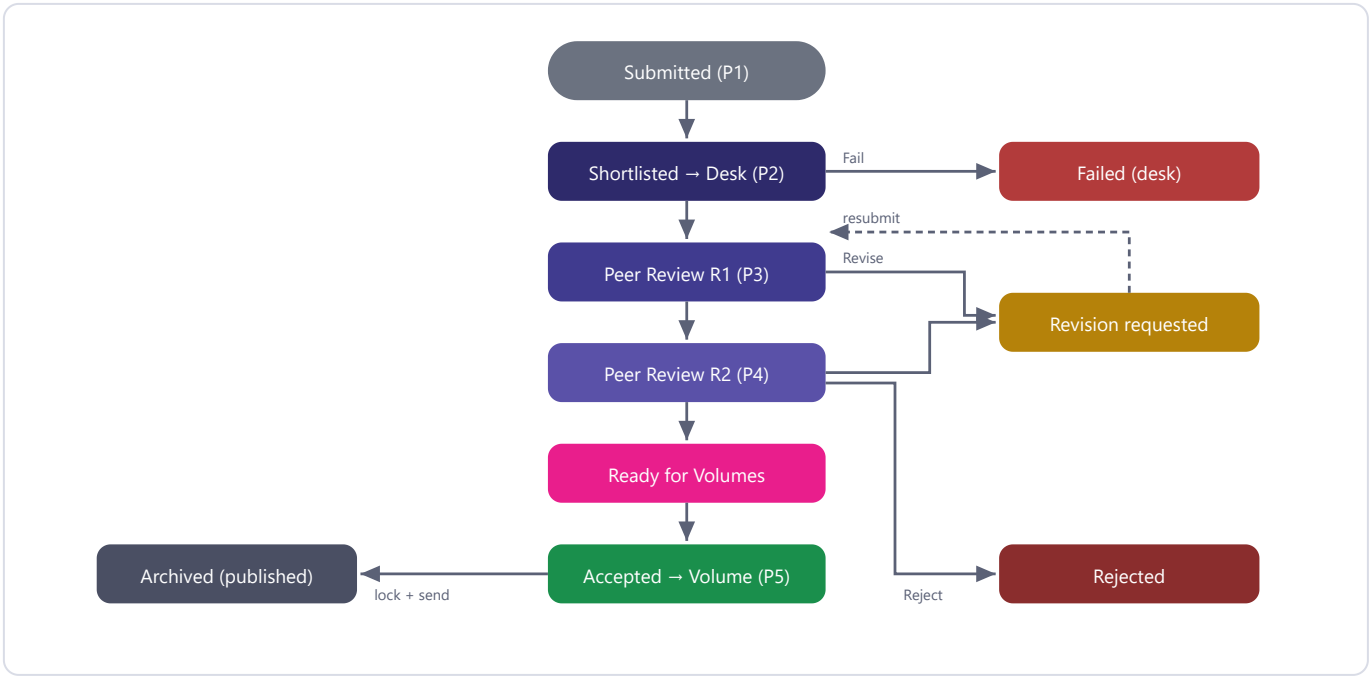
Area	Requirement
Password storage	Hash with argon2id or bcrypt; never store plaintext
Password policy	Raise the prototype "min 6" to ≥10 chars + breached-password check PROPOSED
Email domain	Restrict to institutional domains; verify before first login PROPOSED
Sessions	httpOnly + Secure + SameSite cookies; sensible expiry; server-side invalidation
Abuse prevention	Rate-limit login/register/reset; generic error messages; optional CAPTCHA
Privilege	Self-registration grants **Submitter only** — never SPOC/Editor/Reviewer/Admin
Password reset	Tokenised, single-use, expiring links PROPOSED (no reference screen)

Open decisions

Is there an email-verification step? An Admin approval gate? A forgot-password screen? None appear in the references — all are proposed and must be confirmed (see §18).

7 Complete Paper Lifecycle & Business Rules

A paper moves through clearly-owned stages. Each transition is triggered by a specific role, may require information, generates an activity entry, and is written to an append-only audit log.



Proposed paper lifecycle state machine. Solid = confirmed transitions; dashed = proposed resubmission loop (needs sign-off).

7.1 Status-transition table

From	To	Who	Needs	Notifies	Reason req.?	Reversible?
Submitted (P1)	Shortlisted→Desk (P2)	SPOC/Admin	Grade set	Submitter, Desk	No	Admin only
Submitted (P1)	(remains / not shortlisted)	SPOC	—	—	No	—
Desk (P2)	Peer R1 (P3)	Desk Rev.	Turnitin recorded, Pass	Editor/Admin	No	Admin only
Desk (P2)	Failed	Desk Rev.	Turnitin recorded	Submitter, Admin	Proposed	Admin only
Peer R1 (P3)	Peer R2 (P4)	Peer R1	Decision=Accept	Editor/Admin	No	Admin only
Peer R1/R2	Revision requested	Peer/Editor	Comments	Submitter	Yes	Yes
Peer R2 (P4)	Ready for Volumes	system	R1 & R2 = Accept	Editor	No	auto
Peer R1/R2	Rejected	Peer/Editor	Comments	Submitter, Admin	Yes	Proposed
Ready	Accepted → Volume (P5)	Editor	Volume placement	Submitter	No	before lock
Volume	Locked → Archived	Editor	Volume full/locked	recipients	No	Restore (Admin)

7.2 Core business rules

1. **Auto-entry:** a new paper enters Phase 1 automatically; only its assigned SPOC/reviewers + Admin can see it.
2. **Edit lock:** a submitter may edit the Drive link & themes only until shortlisting begins; then edits lock.
3. **Grade gate:** Phase-1 shortlist is "by Grade" — SPOC sets/confirms the grade first.
4. **Both-rounds rule:** a paper becomes "Ready for Volumes" only when **both** R1 and R2 are Accept.
5. **Separation of duties:** nobody holds a review/grade/edit role on a paper they submitted — blocked at assignment time.

6. **Volume rule:** target 10 papers per volume, 8 volumes; "Fill smartly" auto-fills from the Ready queue; locked/archived volumes are read-only.
7. **Non-destructive history:** every review/decision is a **new record**; prior rounds are never overwritten (see §9).

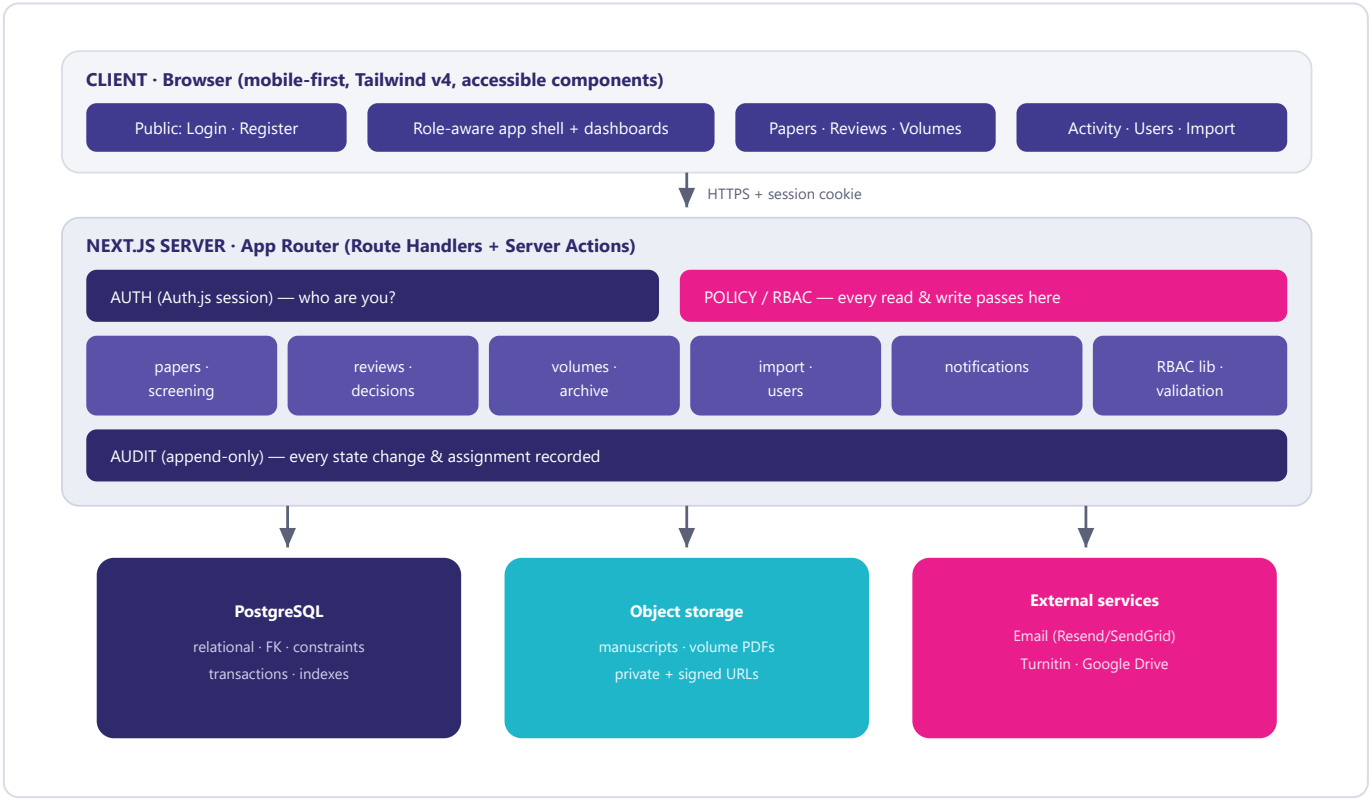
Alternate & edge paths

Incomplete submissions (held in P1), desk failure, reviewer conflict (blocked assignment), overdue reviews (open-slot alerts), revision rounds, rejection, and archive restore. The resubmission loop and withdrawal are PROPOSED and need business sign-off.

8 System Architecture Diagrams

8.1 Recommendation

A **modular monolith**: one Next.js App Router deployment containing both the UI and the secure server logic, talking to a managed PostgreSQL database, private object storage, and a few external services. Microservices are **not** justified at this scale and would add distributed-transaction complexity for no benefit.



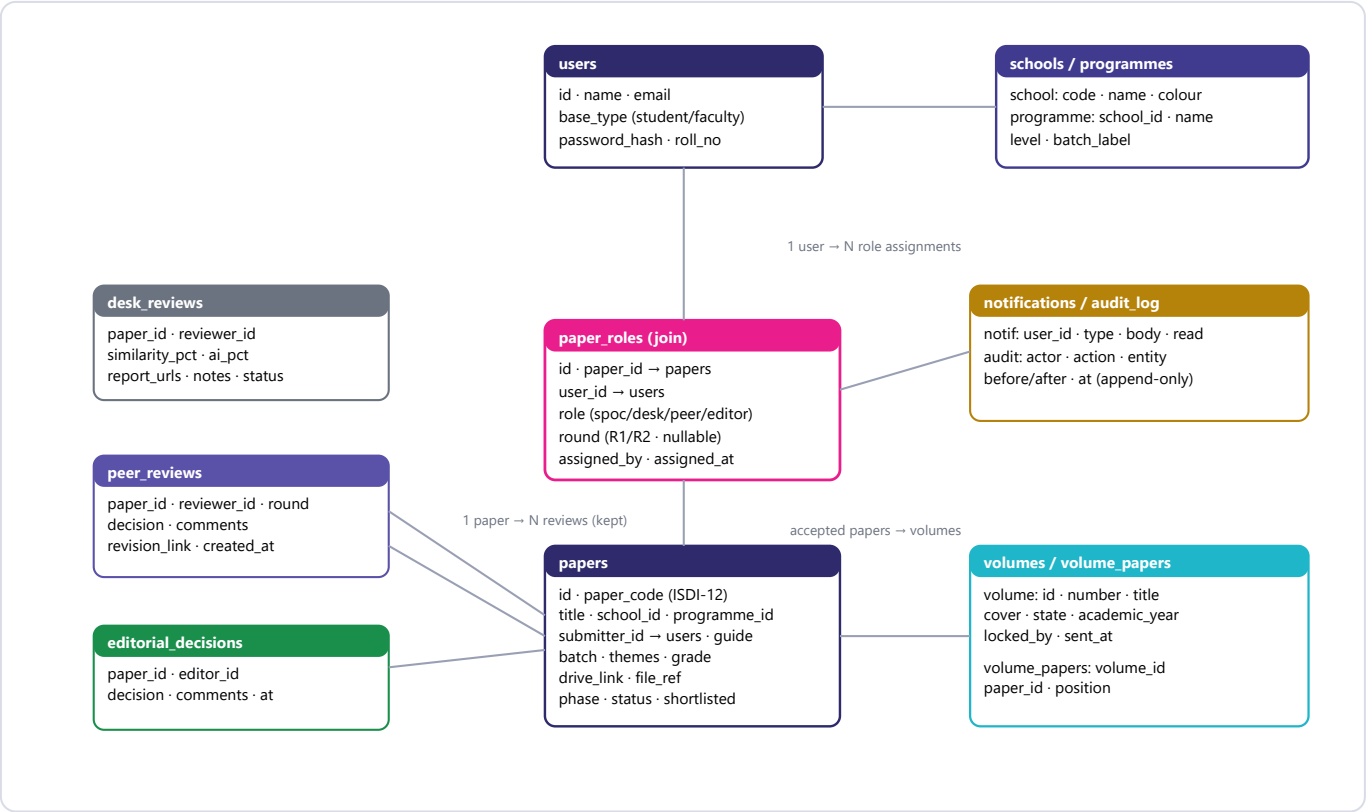
High-level system architecture. The POLICY/RBAC band is the security heart: nothing reaches the data tier without passing it.

8.2 Request round-trip (plain English)

1. Browser sends a request with its session cookie.
2. Server resolves the session — *authentication* (who you are).
3. Policy layer answers "*may this user do this action on this specific paper?*" — *authorization*.
4. If allowed, the data layer runs queries/transactions and appends to audit + activity.
5. File links become signed, time-limited URLs (or the stored Drive link).
6. The server returns **only** the fields this user may see (e.g. a peer reviewer never receives the co-reviewer's report in the payload).
7. The browser renders the result responsively and accessibly.

9 Database Design & ER Diagram

A relational design (PostgreSQL) chosen for integrity and transactions. The central idea mirrors the access model: a **join table of per-paper role assignments** connects people to papers with a role.



Entity-relationship overview. Reviews are append-only child rows, so a new round never overwrites the last.

9.1 Key tables

Table	Purpose	Key relationships / notes
users	Accounts	PK id; unique email; base_type student/faculty; roll_no (students)
schools / programmes	Taxonomy	programme.school_id → schools; seed data
papers	The submission	FK submitter_id, school_id, programme_id; unique paper_code; phase/status enums
paper_roles	**Per-paper RBAC**	FK paper_id + user_id; role enum; round for peer; unique (paper,role,round); conflict: user ≠ submitter
desk_reviews	Turnitin screening	FK paper_id, reviewer_id; similarity/ai %, URLs, status PENDING/PASS/FAIL
peer_reviews	Blind reviews (append-only)	FK paper_id, reviewer_id; round R1/R2; decision; comments; one active per (paper,round)
editorial_decisions	Final decision history	FK paper_id, editor_id; decision; comments
volumes / volume_papers	Publication assembly	state Ready/Placed/Locked/Archived; volume_papers M:N papers↔volumes
notifications	Activity feed	FK user_id; type ROLE/INFO/DECISION/ASSIGNMENT; read flag
audit_log	Accountability	Append-only; actor, action, entity, before/after, timestamp

9.2 Data-integrity principles

- **History is additive.** A revision or a new review round inserts a **new row**; the earlier review stays intact for audit and appeals. Overwriting would destroy the record of why a decision was made.

- **Transactions** wrap multi-table critical actions — submission, shortlist, recording a decision, locking a volume — so they either fully succeed or fully roll back.
- **Files are not blobs in the DB.** Store a reference (Drive link or object-storage key); keep the binary in object storage with signed URLs.
- **Multiple authors / reviewers / rounds** are modelled as rows in child/join tables, not comma-strings — except where the prototype currently uses a single author field (flagged for co-author support).

10 API & Module Architecture

Business logic lives in server modules; UI components stay presentational. Every endpoint calls the shared policy layer before touching data.

10.1 Representative API operations

Area	Operation (example route handler)	Who
Auth	`POST /api/auth/register` · `/login` · `/reset`	Public
Papers	`POST /api/papers` (create) · `PATCH /api/papers/:id` (edit pre-lock) · `GET /api/papers` (scoped list)	Submitter/SPOC/Admin
Screening	`POST /api/papers/:id/desk` (record Turnitin, Pass/Fail)	Desk Rev./Admin
Reviews	`POST /api/papers/:id/reviews` (round R1/R2 decision)	Peer/Admin
Decisions	`POST /api/papers/:id/decision` (accept/revise/reject)	Editor/Admin
Roles	`POST /api/papers/:id/roles` (assign, conflict-checked) · `DELETE …`	Admin (SPOC subset)
Volumes	`POST /api/volumes` · `/:id/fill` · `/:id/lock` · `/:id/send`	Editor/Admin
Import	`POST /api/import` (CSV/Excel upsert on paper_id, dry-run preview)	SPOC/Admin
Users	`GET/POST /api/users` · `PATCH /api/users/:id`	Admin
Notifications	`GET /api/notifications` · `POST /:id/read`	Owner

10.2 Internal modules

auth — session, registration, reset

rbac — `can(user,action,paper)`, visibility, conflict, field redaction

papers — create/edit/list, phase/status machine

screening — desk/Turnitin capture + Pass/Fail

reviews — peer R1/R2 append-only

decisions — editor finalisation

volumes — assembly, lock, archive, restore

import — validated CSV/Excel upsert

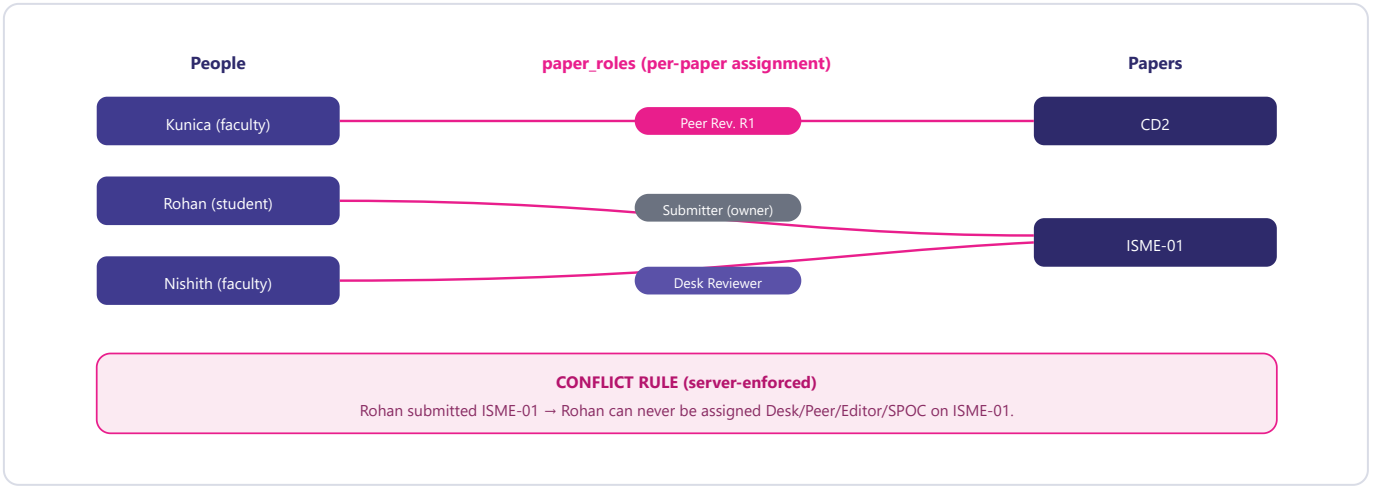
notifications — activity feed + email adapter

audit — append-only change log

11 Authentication & Permission Model

11.1 The per-paper RBAC model

A person's power depends entirely on the **paper** in front of them. The policy layer resolves, for every request: *what role (if any) does this user hold on this paper?*



Roles are edges between a person and a paper. The same person can be a reviewer on one paper and an author on another — but never both on the same paper.

11.2 Visibility & confidentiality rules (confirmed)

Rule	Enforcement
A paper is visible only to its submitter, assigned roles, and Admin	Query filter in policy layer; unassigned access → "no access" screen
Reviewers can't see desk review, co-reviewer's report, or editor's decision	**Field-level redaction at the data layer** — hidden fields never enter the payload
Author identity is visible to every assigned role (single-blind)	Author fields returned to assigned roles only
Nobody reviews/grades/edits their own paper	Conflict check at assignment + at action time

11.3 Policy function (illustrative, JavaScript)

```
// lib/rbac/policy.js  — server only
export async function can(user, action, paperId) {
  if (user.isAdmin) return true;                // admin: every paper, every person
  const roles = await rolesFor(user.id, paperId); // from paper_roles
  const isOwner = await isSubmitter(user.id, paperId);
  switch (action) {
    case 'paper:view':   return isOwner || roles.length > 0;
    case 'paper:edit':   return isOwner && !(await isShortlisted(paperId));
    case 'desk:record':  return roles.includes('desk');
    case 'review:submit':return roles.some(r => r.startsWith('peer'));
    case 'decision:set': return roles.includes('editor');
    case 'role:assign':  return false;           // admin handled above; SPOC subset elsewhere
    default:             return false;           // deny by default
  }
}
// Separation of duties — blocks the "Conflict — can't assign" case:
export async function assertAssignable(userId, paperId) {
  if (await isSubmitter(userId, paperId))
    throw new Conflict("A submitter can't review, grade or edit their own paper.");
}
```

> **Why not just hide the button?**
>
> Hiding UI controls is convenience, not security — anyone can call the API directly. The real control is this server-side function, re-checked on every request, plus field-level redaction so forbidden data never leaves the server.

12 Security Architecture

Threat context

This system holds **unpublished research** and **confidential review information**. A leak could enable plagiarism, compromise blind review, or expose a reviewer. Security is a first-class requirement, not a finishing touch.

Control	Approach
Authentication	Hashed passwords (argon2id/bcrypt); secure session cookies; optional SSO later
Authorization (RBAC)	**Server-enforced** per-paper policy on every request; deny-by-default; Admin override explicit
Least privilege	Registration grants Submitter only; privileged roles assigned per paper by Admin
Separation of duties	Hard conflict block (no role on own paper) at assignment & action time; logged
Reviewer confidentiality	Field-level redaction; blind-review data never serialised to the wrong role
Registration abuse	Rate limiting; institutional-domain restriction; email verification
Session security	httpOnly/Secure/SameSite; rotation on privilege change; idle + absolute timeouts
Input validation	Server-side schema validation (e.g. Zod) on every mutation — never trust the client
SQL injection	Parameterised queries via the ORM; no string-built SQL
XSS	React auto-escaping; sanitise any rich text; strict CSP
CSRF	SameSite cookies + CSRF tokens / same-origin checks on mutations
Rate limiting	On auth, reset, import and other sensitive endpoints
Secure file handling	Validate type/size/content; private storage; signed, time-limited, access-checked downloads; AV scan
Secrets	Env vars / secret manager; never committed; rotated
Error handling	Generic user-facing errors; detailed logs server-side only
Audit trail	Append-only log of every state change & role assignment (incl. blocked ones)
Backups	Encrypted, tested restores; access-controlled

12.1 Privileged access governance

Admin is the only role that can create/alter other roles. First Admin is seeded; further Admin promotions should require audit logging and ideally a second-person approval PROPOSED. No self-service path may ever yield a privileged role.

12.2 Why "hiding a button" is not security

UI hiding only changes what a cooperative browser shows. An attacker calls the API directly. Therefore every rule in §11 is re-enforced server-side, and the API returns only permitted fields. The frontend's role-awareness is for *usability*; the server's policy layer is for *security*.

13 Responsive Design Strategy

Mobile-first with Tailwind v4. The supplied designs are desktop-oriented; this section defines how each pattern adapts down to phone width — a desktop screenshot does not define the mobile experience.

Pattern	Phone (≤640px)	Tablet (641–1024px)	Desktop (>1024px)
Sidebar nav	Hamburger → slide-over drawer	Collapsible rail	Fixed sidebar (as designed)
Papers table	Stacked cards (ID + title + status chips)	Reduced columns + horizontal scroll	Full table
Filters	"Filters" button → bottom sheet	Inline wrap	Inline row
KPI cards	1–2 per row, scroll	3 per row	6 in a row
Submit / long forms	Single column, grouped steps, sticky submit	Single column wider	Two-column where shown
Modals (assign role, new paper)	Full-screen sheet	Centered dialog	Centered dialog
Volumes board	Vertically stacked columns	Horizontal scroll	Kanban columns

13.1 Accessibility & states

- Semantic HTML, ARIA where needed, visible focus rings, keyboard-navigable flows, ≥44px touch targets.
- Contrast checked — the vivid school colours (pink/red) need care on text; use as chips/accents, not body text.
- Every data view has explicit **loading**, **empty** ("No papers match."), **error**, and **success** states.
- Consistency via reusable components + Tailwind v4 `@theme` design tokens (brand + school colours, spacing, radius).

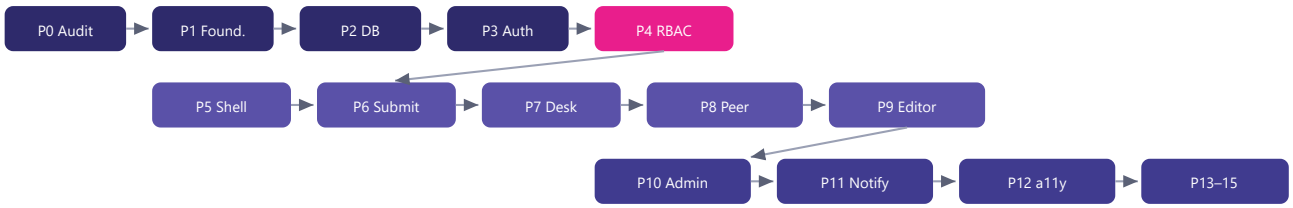
14 Recommended Next.js Folder Structure

App Router, JavaScript only. Route groups organise by role for clarity, but **route groups are not security** — the policy layer is. Business logic sits under `lib/`, kept out of presentational components.

```
research-publication-portal/
├─ app/
│  ├─ layout.js                # root shell, fonts, providers
│  ├─ globals.css              # Tailwind v4 @theme tokens (brand + schools)
│  ├─ (public)/
│  │  ├─ login/page.js
│  │  └─ register/page.js
│  ├─ (app)/                   # authenticated shell (sidebar, role switcher)
│  │  ├─ layout.js
│  │  ├─ dashboard/page.js
│  │  ├─ papers/
│  │  │  ├─ page.js            # scoped list
│  │  │  └─ [id]/page.js       # paper detail (tabs: overview/access/history)
│  │  ├─ submit/page.js        # submitter
│  │  ├─ import/page.js        # spoc/admin
│  │  ├─ volumes/page.js       # editor/admin
│  │  ├─ archive/page.js       # editor/admin
│  │  ├─ users/page.js         # admin
│  │  └─ activity/page.js
│  └─ api/
│     ├─ auth/[...]/route.js
│     ├─ papers/route.js  ·  papers/[id]/route.js
│     ├─ papers/[id]/{desk,reviews,decision,roles}/route.js
│     ├─ volumes/[...]/route.js  ·  import/route.js  ·  users/route.js
│     └─ notifications/route.js
├─ components/
│  ├─ ui/            # Button, Badge, Chip, Table, Modal, Field (reusable)
│  ├─ layout/        # Sidebar, RoleSwitcher, ScopeBanner, KpiCard, PhaseTimeline
│  └─ papers/        # PapersTable, PaperDetail, ReviewForm, DeskPanel, AssignRoleModal
├─ lib/
│  ├─ auth.js            # Auth.js config
│  ├─ db.js              # Prisma client
│  ├─ rbac/policy.js     # can(user, action, paper)
│  ├─ rbac/visibility.js # query scoping + field redaction
│  ├─ validation/        # Zod schemas per entity
│  ├─ services/          # papers, reviews, volumes, import, notifications
│  ├─ files.js           # signed URLs / Drive link handling
│  └─ constants.js       # roles, phases, statuses, schools
├─ prisma/
│  ├─ schema.prisma  ·  seed.js  ·  migrations/
├─ tests/           # unit · integration · e2e (incl. cross-role access)
├─ public/          # ATLAS logo assets, favicon (square tile)
└─ .env(.local)     # secrets — never committed
```

Folder	Holds
`app/(app)/`	Authenticated pages behind the role-aware shell
`app/api/`	Route handlers — each calls `lib/rbac` before data access
`components/`	Presentational, reusable UI (no business rules)
`lib/rbac/`	The single source of truth for permissions & visibility
`lib/services/`	Business logic & transactions, DB-aware
`prisma/`	Schema, seed, migrations

15 Development Roadmap



Phase dependency map. The critical path P0→P1→P2→P3→P4 (RBAC) gates every role feature.

Phase	Objective	Key acceptance	MVP?
P0 Audit & sign-off	Lock scope, answer open questions	Stakeholders approve matrix & lifecycle	✓
P1 Foundation	Next.js/Tailwind skeleton + brand tokens	Builds, lints, no TS	✓
P2 Database	Schema with non-destructive history	Migrations run; ERD matches	✓
P3 Auth & accounts	Register/login/session	Verified flow; rate-limited	✓
P4 RBAC **(highest risk)**	Central policy + visibility + conflict	Cross-role leakage tests pass	✓
P5 Shell & dashboards	Role-aware nav, KPIs, timeline	Correct nav per role; mobile drawer	✓
P6 Submission	Create/track, edit-lock	Submit→P1; lock at shortlist	✓
P7 Desk screening	Turnitin capture, Pass/Fail	Advance/fail correct; data hidden	✓
P8 Peer review	Blind R1/R2 append-only	Both-accept rule; redaction	✓
P9 Editorial & volumes	Finalise + assemble volumes	Lock/archive auditable	✓
P10 SPOC & Admin	Shortlist, assign, users, import	Conflict-safe assignment; import audited	partial
P11 Notifications & audit	Activity + email + audit log	Events fire; log append-only	basic
P12 Responsive & a11y	Phone→desktop, WCAG AA	Usable at 360px; keyboard flows	Later
P13 Testing	Unit/integration/E2E/access	Green suite; no leakage	Later
P14 Perf & deploy	Tune + deploy to Node host	Reproducible deploy	Later
P15 Prod verify/backup	Smoke, backups, monitoring	Tested restore; alerts live	Later

MVP recommendation

Deliver P0–P9 plus the SPOC/Admin essentials of P10 (shortlist, conflict-checked assignment, people & roles) and a basic activity feed — one full cycle end-to-end. Defer bulk import, Turnitin API, SSO, advanced analytics, archive restore and PDF export. Do

not

build withdrawal, co-author UI or automated resubmission until confirmed.

16 Testing Strategy

Layer	What it covers
Unit	Policy function, validators, status-machine transitions, grade/prefix inference
Integration	API routes + DB (create paper, record desk, submit review, lock volume) incl. transactions
End-to-end	Full journey: register → submit → shortlist → desk → R1 → R2 → accept → volume → archive
Authorization / cross-role	Every role tries every forbidden action & forbidden paper → must be denied; payload never leaks hidden fields
File up/download	Type/size/content validation; signed-URL access control; unauthorised download blocked
Lifecycle transitions	Only legal transitions allowed; conflict block enforced; both-accept rule
Responsive UI	Breakpoint checks at 360/768/1280; drawer, stacked tables, sheets
Accessibility	Automated (axe) + keyboard-only + contrast spot-checks (WCAG 2.1 AA)
Performance	List pagination, query/index checks under realistic data (~1,000 papers)
Backup/restore	Scheduled backup + periodic restore drill verification

Highest-value tests

The cross-role authorization suite is the most important: it is what proves the confidentiality promises ("hidden from reviewers…", "nobody can review their own paper") actually hold at the API, not just in the UI.

17 Deployment & Maintenance

> **XAMPP cannot host this in production**
>
> XAMPP (Apache + PHP + MySQL) is only the local folder location on this machine. A Next.js production server needs a **Node.js**
> runtime. Do not assume the existing PHP hosting can run it — verify hosting capabilities and choose a Node-capable target.

17.1 Environments

Environment	Runtime	Data
Local dev	Next.js dev + Node; Postgres via Docker or dev cloud DB	Seed / synthetic
Staging	Mirror of prod	Anonymised; email in test mode
Production	Node host (Vercel / container / VPS / institutional Node) + managed Postgres + object storage	Live; real email keys; backups

17.2 Operations checklist

- **Config:** all secrets in env vars / secret manager (DB URL, auth secret, storage & email keys). Never commit secrets or real data.
- **Migrations:** run via Prisma in CI/CD; forward-only, reviewed.
- **Deploy checks:** build, lint, test gate; smoke test after deploy.
- **Backups:** automated encrypted daily DB backups + object-storage backup; tested restore; documented RTO/RPO.
- **Rollback:** previous build re-deployable; migration rollback plan.
- **Monitoring:** error tracking (e.g. Sentry), uptime, DB metrics, audit-log review.

18 Risks, Dependencies & Unresolved Questions

18.1 Top risks

Risk	Impact	Mitigation
Per-paper RBAC implemented inconsistently	Data leak / broken blind review	Single policy layer; deny-by-default; cross-role test suite
Google Drive links are hard to secure/audit	Unpublished research exposed	Move to private object storage + signed URLs
Email mocked at launch	Users miss status changes	Wire Resend/SendGrid before go-live; in-app feed as fallback
Hosting mismatch (XAMPP assumption)	Cannot deploy	Confirm Node host early (G1)
Ambiguous revision/withdrawal rules	Rework late	Resolve in P0 before DB freeze

18.2 Unresolved questions (priority) — see `open-questions-and-assumptions.md`

#	Question	Pri
A1	Exact Admin↔SPOC split of reviewer-assignment authority	P0
A4	How are additional Admins provisioned & changed	P0
C1	Google Drive links vs private object-storage uploads	P0
C6	Exact phase/status enum & every allowed (and reversible) transition	P0
G1	Production hosting target (must be Node, not XAMPP)	P0
A2	Single-blind (as evidenced) vs double-blind author masking	P1
B1–B4	Email verification, approval gate, password reset, domain/password policy	P1
C2/C3	Co-author support; revision/resubmission loop & round cap	P1
C7/D2	Grade list + shortlist cut-off; Turnitin threshold rules	P1
E1–E3	"Send for publishing" handoff; 10×8 hard vs target; archive restore rules	P1
C4	Withdrawal flow (if any)	P2

19 Glossary of Technical Terms

Term	Plain-English meaning
Frontend	The part you see and click in the browser.
Backend	The server-side "back office" that checks rules and does the work.
Database	Organised storage for all the application's information.
Authentication	Proving who you are (login).
Authorization	Deciding what you're allowed to do.
RBAC	Role-Based Access Control — permissions decided by your role; here, **per paper**.
API	The messaging channel between the screen and the server.
Route Handler	A small server function in Next.js that answers an API request.
Server Action	A Next.js function that runs on the server when a form is submitted.
ORM / Prisma	A tool that lets code talk to the database safely in JavaScript.
Transaction	A group of database changes that all succeed or all fail together.
Object storage	A secure file store (for manuscripts/PDFs) separate from the database.
Signed URL	A temporary, permission-checked link to download a private file.
Single-blind review	Reviewers don't see each other/the decision; the author's name is visible to assigned roles.
SPOC	School/Specialisation Point of Contact — owns Phase-1 shortlist & grading.
Desk review	Phase-2 screening: plagiarism/AI check via Turnitin, then Pass/Fail.
Volume	A published collection of ~10 accepted papers.
Audit log	An unchangeable record of who did what, when.
Upsert	Insert a new record or update the matching existing one (import keys on Paper ID).
WCAG 2.1 AA	A widely-used accessibility standard.

20 Final Implementation Checklist

Before coding (Phase 0)

- ☐ Sign off the role matrix, lifecycle & transition table (§5, §7).
- ☐ Resolve P0 open questions: Admin↔SPOC assignment (A1), Admin provisioning (A4), storage (C1), transitions (C6), hosting (G1).
- ☐ Confirm auth policy: verification, approval, reset, domain/password rules (B1–B4).
- ☐ Confirm manuscript storage & email provider; obtain keys or accept mock for pilot.

Foundations (P1–P4)

- ☐ Next.js App Router (JS only), Tailwind v4 brand + school tokens, favicon from square tile.
- ☐ PostgreSQL schema with append-only review history; migrations + seed (schools/programmes).
- ☐ Auth.js register/login/session; rate limiting; Submitter-only self-registration.
- ☐ Central `rbac` policy + visibility redaction + conflict block; **cross-role tests green**.

Features (P5–P11)

- ☐ Role-aware shell, dashboards, scope banners, role switcher.
- ☐ Submission + edit-lock; desk/Turnitin; blind R1/R2; editorial decision; volumes/archive.
- ☐ SPOC shortlist/assign/import; Admin people & roles; activity + email + audit.

Quality & launch (P12–P15)

- ☐ Responsive + WCAG AA; full test suite; perf tuning.
- ☐ Deploy to Node host (not XAMPP); backups + tested restore; monitoring.

> **Status & next step**
>
> This blueprint and its four companion Markdown files are complete and mutually consistent.
> **No application has been implemented.**
> Recommended next phase:
> **Phase 0 — requirement approval**
> , resolving the P0 open questions above. Please review and approve before any coding begins.